

Object Oriented Programming part 3

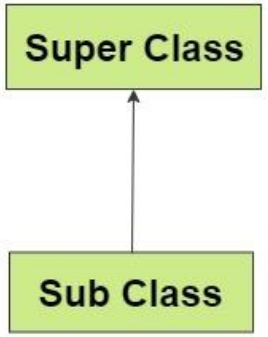
Inheritance

Dr Raju Nadimpalli

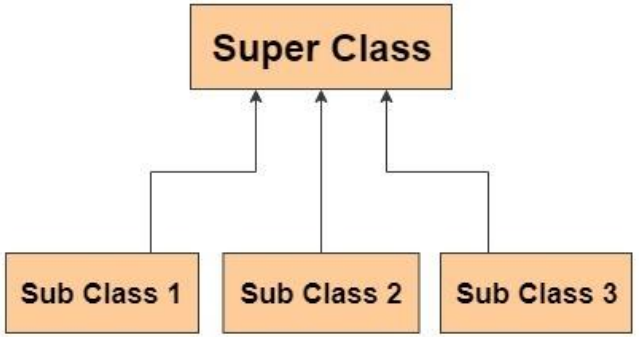
Inheritance in Python

- A Powerful feature of object-oriented programming is the **ability to create a new class by extending an existing class**.
- It refers to **defining a new class with little or no modification** to an existing class.
- The new class is called derived (or child) class and the one from which it inherits is called the base (or parent) class.
- Code reusability being the forte of inheritance

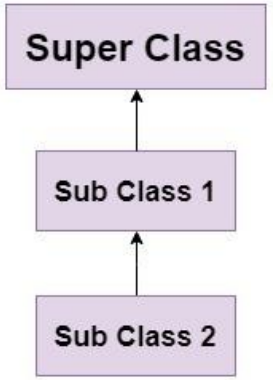
Single Inheritance



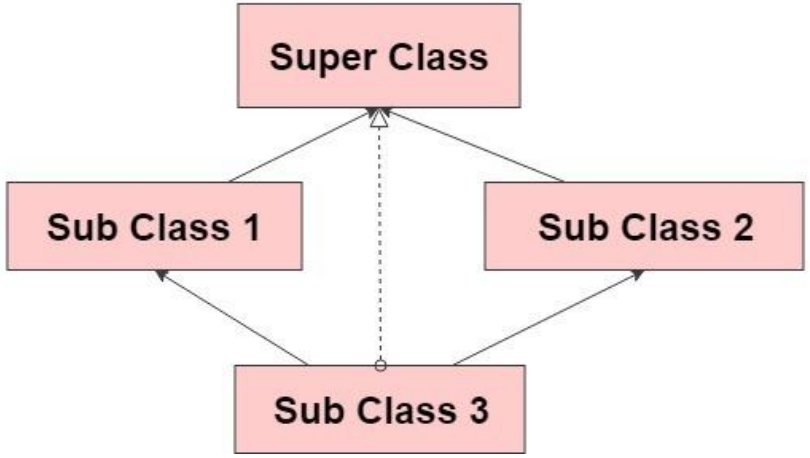
Hierarchial Inheritance



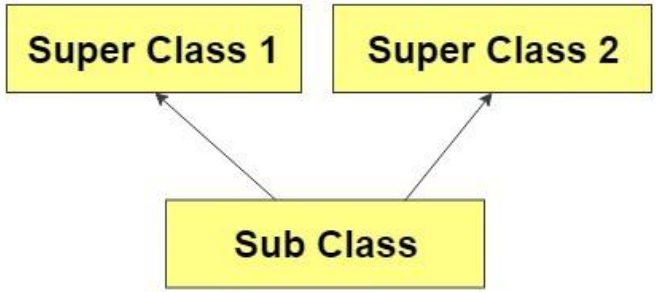
MultiLevel Inheritance



Hybrid Inheritance



Multiple Inhertance



Inheritance

- **Syntax for inheritance:**

```
class BaseClass:
  Body of base class
class
DerivedClass(BaseClass):
  Body of derived class
```

- Derived class inherits features from the base class where new features can be added to it.
- This results in re-usability of code.

Note:

- All classes, except the special class ***object***, are derived classes, even if they do not have a base class name.
- The ***object*** class is the only class that is not derived, since it is the base of the inheritance hierarchy.

Example for single inheritance

```
class Parent:
    def __init__(self, a):
        self.a = a

    def method1(self):
        print(self.a)
```

```
class Child (Parent):
    def __init__(self, a, b):
        self.a = a
        self.b = b

    def method2(self):
        print(self.a + self.b)
```

```
p = Parent('Hai')
p.method1()

print()

c = Child("Hai ", “Python Online!")
c.method1()
c.method2()
```

Hai

Hai
Hai Python Online**!**

Method Overriding

```
class Parent:
    def __init__(self, a):
        self.a = a

    def method1(self):
        print(self.a)

class Child (Parent):
    def __init__(self, a):
        self.a = a

    def method1(self):
        print(self.a * 2)

p = Parent('Hai')
p.method1()

c = Child("Naresh Tech")
c.method1()
```

Note:

- **Method signature refers** to the method name, order and the total number of its parameters.
- When a method in a subclass has the ***same name, same parameters or signature and same return type(or sub-type) as a method in its super-class,*** then the **method in the subclass is said to override the method in the super-class.**

```
Hai

Naresh Tech Naresh Tech Naresh Tech
```

Accessing the Inherited Variables and Metho

- Single inheritance enables a derived class to inherit variables and methods from a single base class.
- This includes __init__() method. So, if you do not define it in a derived class.

```
class Student:
    def __init__(self, Number, Name, Test1, Test2):
        self.number = Number
        self.name = Name
        self.test1 = Test1
        self.test2 = Test2

    def displaystud(self):
        print(f"{self.number} is the regd. number of student
'{self.name}'")

class Marks(Student):
    def displaymarks(self):
        print(f"{self.test1} and '{self.test2}' are test scores")
```

```
stud_marks =  Marks(100,"Sachin", 88,
99)
stud_marks.displaystud()

stud_marks.displaymarks()
```

```
100 is the regd. number of student

'Sachin'

88 and '99' are test scores
```

super() for accessing base class constru

- In a single inheritance, the built-in *super()* function can be used to refer to base classes.
- If we want to access the data attributes from the base class in addition to the data attributes being specified in the derived class's __init__() method,

  then we must explicitly call the base class __init__() method using *super()*
- Syntax : super().__init__(base_class_parameter(s))

Note:

- The derived class __init__() method contains its own parameters along with the parameters specified in the __init__() method of base class.

super() for accessing base class constructor

```
class Student:
    def __init__(self, Number, Name):
        self.number = Number
        self.name = Name

    def displaystud(self):
        print(f"{self.number} is the regd. number of student
'{self.name}'")

class Marks(Student):
    def __init__(self, Number, Name, Test1, Test2):
        super().__init__(Number, Name)
        self.test1 = Test1
        self.test2 = Test2

    def displaymarks(self):
        print(f"{self.test1} and '{self.test2}' are test scores")
```

```
stud_marks =  Marks(100,"Sachin", 88,
99)
stud_marks.displaystud()
stud_marks.displaymarks()
```

100 is the regd. number of student 'Sachin'
88 and '99' are test scores

super() for accessing base class methods

- The super() function is useful for accessing the base class methods that have been overridden in a derived class without explicitly specifying the base class name.
- Syntax: super().invoke_base_class_method(argument(s))

```
class Book:
    def __init__(self, author, title):
        self.author = author
        self.title = title
    def book_info(self):
        print(f"{self.title} is authored by {self.author}")

class Fiction(Book):
    def __init__(self, author, title, publisher):
        super().__init__(author, title)
        self.publisher = publisher

    def book_info(self):
        print(f"{self.title} by {self.author} published
{self.publisher}")

    def invoke_base_class_method(self):
        super().book_info()
```

```
book1 = Fiction("Daniel Silva", "Prince of Fire", "Berkley")
book1.book_info()
book1.invoke_base_class_method()
print("------------------------------")
print("Base Class")
book2 = Book("Lee Child", "One Shot")
book2.book_info()
```

```
Prince of Fire by Daniel Silva published
Berkley
Prince of Fire is authored by Daniel Silva
------------------------------
Base Class
One Shot is authored by Lee Child
```

Overloading

- Overloading is the ability to define a function with the same name multiple times.
- The definitions are different concerning the number of parameters and types of the parameters.
- It's the ability of one function to perform different tasks, depending on the number of parameters or the types of the parameters.

```
def f(n):
    return n + 10

def f(n,m):
    return n + m + 10

print(f(3, 4))
```

17

Override

- When we override a method, we sometimes want to reuse the method of the parent class and at some new stuff.

```
class Parent:
    def __init__(self, a):
        self.a = a

    def method1(self):
        print(self.a)

class Child (Parent):
    def __init__(self, a):
        self.a = a

    def method1(self):
        print(self.a * 2)
```

```
p = Parent('Hai')
p.method1()

c = Child("Python Online!")
c.method1()
```

Multiple Inheritance

- Python supports a form of multiple inheritances.
- A derived class definition with multiple base classes.
- Syntax: ***class DerivedClassName(Base_1, Base_2, Base_3):***

  <***statement-1***>

  .....

  <***statement-N***>

- Derived class *DerivedClassName* is inherited from multiple base classes, *Base_1*, *Base_2*, *Base_3*.

Note:

- The search for attributes inherited from a parent class as depth-first, left-to-right.

Multiple Inheritance

```
class Addition:
    def Sum(self,a,b):
        return a+b;

class Multiplication:
    def Mul(self,a,b):
        return a*b;

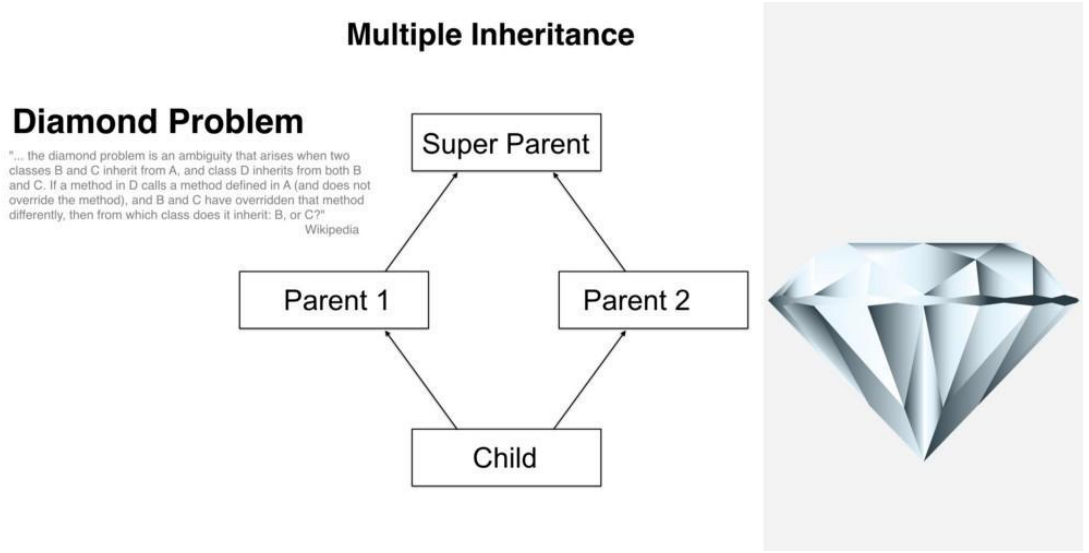
class Derived(Addition,Multiplication):
    def Divide(self,a,b):
        return a/b;

d = Derived()
print(d.Sum(10,20))
print(d.Mul(10,20))
print(d.Divide(20,10))
```

```
30
200
2.0
```

Method Resolution Order

- Python supports multiple inheritance, what means, in Python, a class can inherit features and attributes from multiple classes.
- MRO or Method Resolution Order is the hierarchy in which base classes are searched when looking for a method in the parent class.



Method Resolution Order

- Python uses c3 linearization algorithm which is also called as c3 superclass linearization.

C3 linearization

The C3 superclass linearization of a class is the sum of the class plus a unique merge of the linearizations of its parents and a list of the parents itself.

- i.e. that if we have same structure in new type classes — the method resolution should be D > B > C > A .

Method Resolution Order

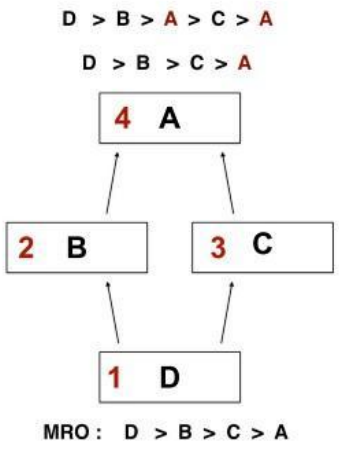
New Style Classes

```
class A(object):
    pass

class B(A):
    pass

class C(A):
    pass

class D(B, C):
    pass
```



C3 Linearization Algorithm

- C3 linearization algorithm enforces following 2 constraints

1. Children precede their parents

2. If a class inherits from multiple classes, they are kept in the order specified in the tuple of the base class.

Method Resolution Order

```
class A:
    def m(self):
        print("m of A called")

class B(A):
    def m(self):
        print("m of B called")

class C(A):
    def m(self):
        print("m of C called")

class D(B,C):
    pass

x = D()
x.m()  #m of B called
```

```
class A:
    def m(self):
        print("m of A called")

class B(A):
    pass

class C(A):
    def m(self):
        print("m of C called")

class D(B,C):
    pass

x = D()
x.m() # m of C called
```

```
class A:
    def m(self):
        print("m of A called")

class B(A):
    pass
class C(A):
    pass

class D(B,C):
    pass

x = D()
x.m() # m of A called
```

The optimal way to solve the problem, which is the "super" pythonic way, would be calling the super function:

```
class A:
    def m(self):
        print("m of A called")

class B(A):
    def m(self):
        print("m of B called")
        super().m()

class C(A):
    def m(self):
        print("m of C called")
        super().m()

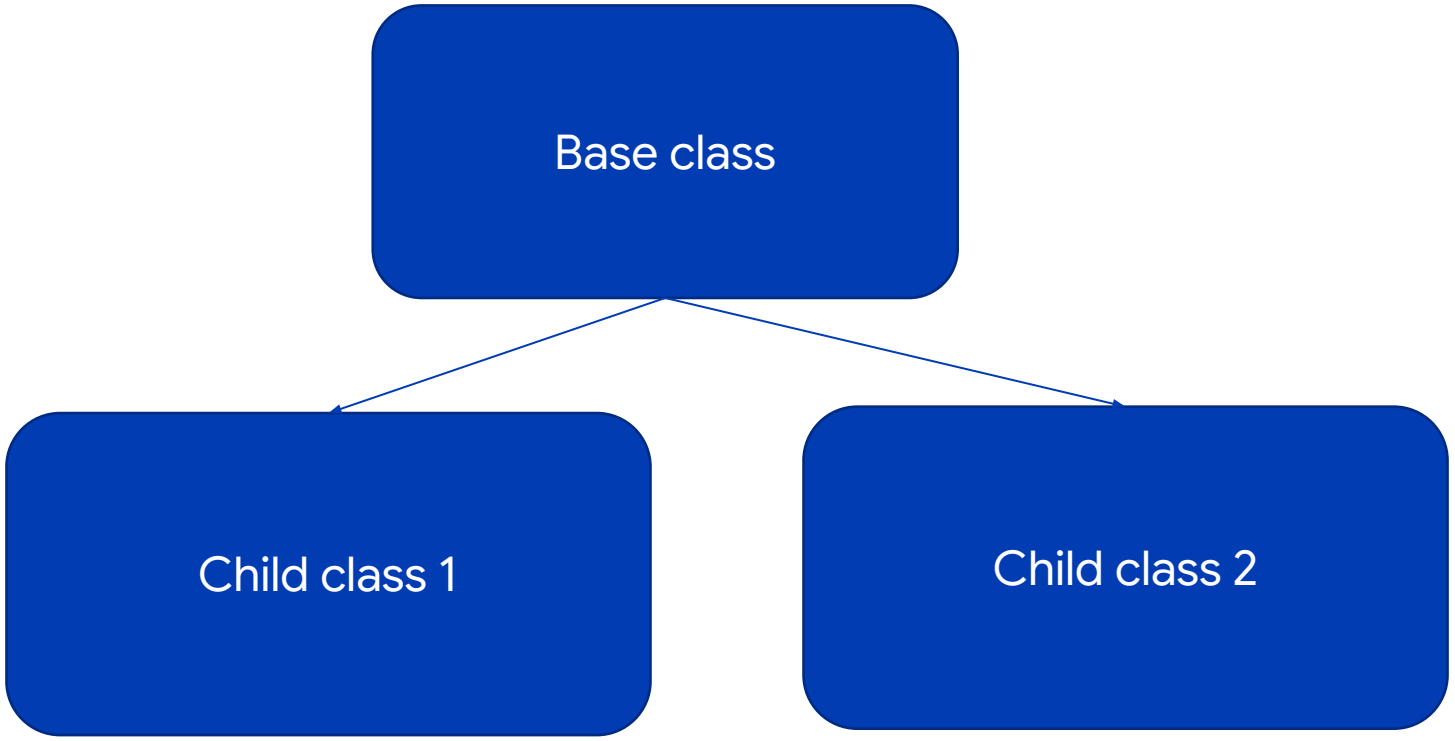
class D(B,C):
    def m(self):
        print("m of D called")
        super().m()
```

```
x = D()
x.m()
print(D.mro())
print(B.mro())
print(C.mro())
print(A.mro())
```

```
m of D called
m of B called
m of C called
m of A called
[<class '__main__.D'>, <class '__main__.B'>, <class
'__main__.C'>, <class '__main__.A'>, <class 'object'>]
[<class '__main__.B'>, <class '__main__.A'>, <class 'object'>]
[<class '__main__.C'>, <class '__main__.A'>, <class 'object'>]
[<class '__main__.A'>, <class 'object'>]
```

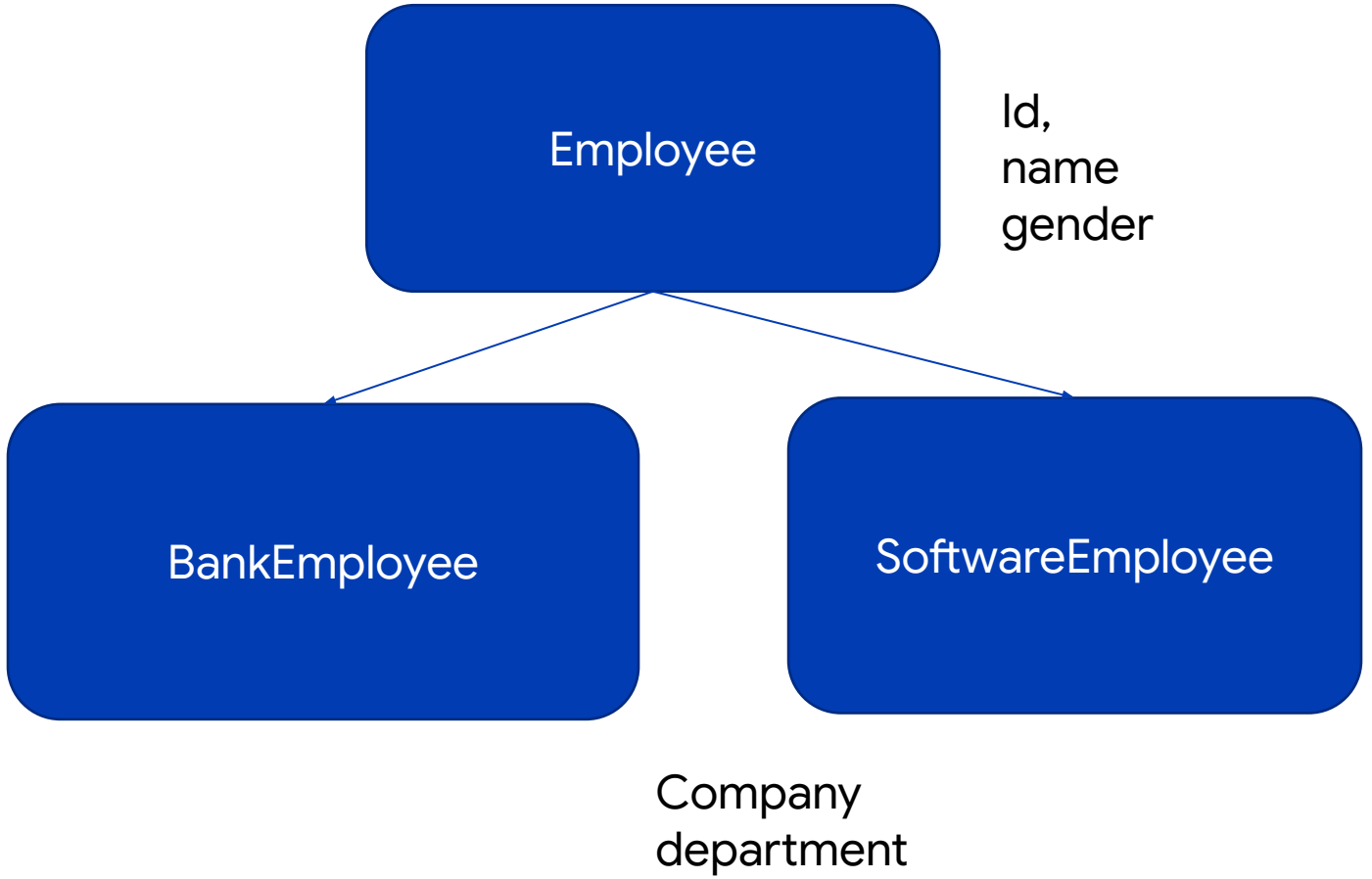
Hierarchical Inheritance

- If more than one class is inherited from the base class, it's known as hierarchical inheritance.
- In hierarchical inheritance, all features that are common in child classes are included in the base class.



Hierarchical Inheritance

```
class Employee:
    def __init__(self):
        self.__id=None
        self.__name=None
        self.__gender=None
    def
setData(self,id,name,gender):
        self.__id=id
        self.__name=name
        self.__gender=gender
    def showData(self):
        print("Id: ",self.__id)
        print("Name: ", self.__name)
        print("Gender: ",
self.__gender)
```



Hierarchical Inheritance

```
class BankEmployee(Employee):
    def __init__(self):
        self.__company=None
        self.__dept=None
    def
setEmployee(self,id,name,gender,comp,dept):
        self.setData(id,name,gender)
        self.__company=comp
        self.__dept=dept
    def showEmployee(self):
        self.showData()
        print("Bank: ", self.__company)
        print("Department: ", self.__dept)
```

```
class SoftwareEmployee(Employee):
    def __init__(self):
        self.__company=None
        self.__dept=None
    def
setEmployee(self,id,name,gender,comp,dept):
        self.setData(id,name,gender)
        self.__company=comp
        self.__dept=dept
    def showEmployee(self):
        self.showData()
        print("Softwre: ", self.__company)
        print("Department: ", self.__dept)
```

Hierarchical Inheritance

```
def main():
    print("Bank Employee Object")
    e1=BankEmployee()
    e1.setEmployee(100,"Sachin","Male","OBC","Manager")
    e1.showEmployee()
    print("\nSoftware Employee")
    e2 = SoftwareEmployee()
    e2.setEmployee(1, "Sourab", "Male", "Info Solutions", "Data Science")
    e2.showEmployee()

if __name__=="__main__":
    main()
```

```
Bank Employee Object
Id:  100
Name:  Sachin
Gender:  Male
Bank:  OBC
Department:  Manager

Software Employee
Id:  101
Name:  Sourab
Gender:  Male
Softwre:  Info Solutions
Department:  Data Science
```

main() in Python

- Most programming languages have a special function called main() which is the execution point for a program file.
- Python interpreter, runs each line serially from the top of the file and has no explicit main() function.
- Python offers other conventions to define the execution point. One of them is using the main() function and the __name__ property of a python file.

Save as HelloWorld.py

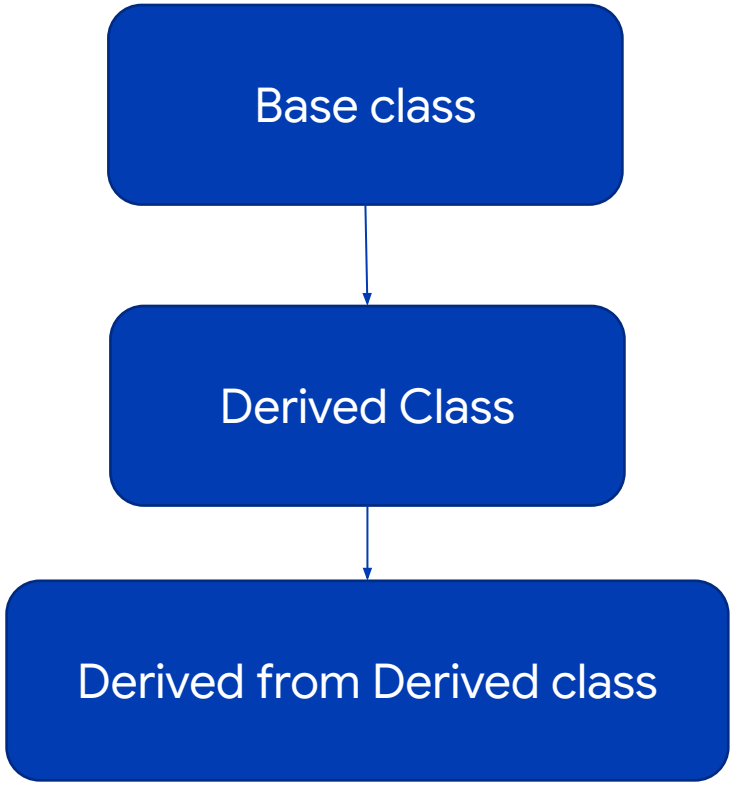
```
def main():
    print("Hello World")
if __name__=="__main__":
    main()
```

- The __name__ variable is a special built-in Python variable that shows the name of the current module.
- Will print as Hello World.
- Here, we have created a custom main() function in the helloworld.py file. It is executed only when the program is run as a standalone script.

Multilevel Inheritance

- A class can also be derived from another derived class.

```
class class1:
    <class-suite>
class class2(class1):
    <class suite>
class class3(class2):
    <class suite>
```



Multilevel Inheritance

```
class A:
    def a(self):
        print("In A")
class B(A):
    def b(self):
        print("In B")
class C(B):
    def c(self):
        print("In C")

c_obj = C()
c_obj.c()
c_obj.b()
c_obj.a()
```

```
In C
In B
In A
```

Abstract methods and classes

- Abstract classes are classes that contain one or more abstract methods.
- An abstract method is a method that is declared but contains no implementation.
- Abstract classes cannot be instantiated and require subclasses to provide implementations for the abstract methods.

Note:

1. In Python abstract class is created by deriving from the *meta class ABC* which belongs to the **abc** (Abstract Base Class) module.
2. For defining abstract methods in an abstract class method has to be decorated with **@abstractmethod** decorator.
3. From **abc** module @**abstractmethod** decorator must

Abstract methods and classes

```
from abc import ABC, abstractmethod

class Computer(ABC):
    @abstractmethod
    def process(self):
        pass

class Laptop(Computer):
     def process(self):
        print("working on Laptop")

class Desktop(Computer):
    def process(self):
        print("working on Desktop")

com = Laptop()
com.process()

com= Desktop()
com.process()
```

```
working on Laptop
working on Desktop
```

Points to be remembered

1. Abstract class can have both concrete methods as well as abstract methods.
2. Abstract class works as a template for other classes. Using an abstract class you can define a generalized structure without providing complete implementation of every method.
3. Abstract class can’t be instantiated, so it is not possible to create objects of an abstract class.
4. Generally abstract methods defined in abstract class don’t have any body, sub classes provide implementation for abstract methods.
5. If any abstract method is not implemented by the derived class Python throws an error.